

플레이어 재설계 청사진

지금까지 짚은 문제들을 하나의 실행 가능한 리팩터 계획으로. 게이팅을 러너로 모으고, 발동을 한 경로로, 연출을 밖으로.

무엇을 왜 — 5개의 문제

#	심각	문제 (현재)	해결 방향
1	●	게이팅이 3곳 분산 — Combat 평타레이트 / Stat Q-E쿨 / Runner 쿨. 쿨 상태가 2곳 중복	쿨 판단을 러너 하나로 통일
2	●	발동 경로 비대칭 — 평타-Q는 애니이벤트, E만 즉발. 단일 경로 목표가 깨짐	전부 AbilityBeat 경로로
3	●	Q가 Timeline 하드결합 + HUD 토글이 캐릭터로 누수 (프레젠테이션)	컷신 연출을 SkillDirector로 분리
4	●	BaseCharacter God object — 컷신·캔버스·오라로드까지 소유	관심사 분리 → 얇은 조율자
5	🐉	PlayerController.Dispose 누수 — SkillQ/E 등 콜백 미해제 → 재진입 시 중복 발동	Register에 대응하는 Unregister

어떻게 — 4개의 원칙

게이트 = 러너 하나

쿨·코스트·태그·캐스트타임 판단의 유일한 권위는 AbilityRunner. 앞단 게이트 제거.

시각 = 주입

현재 시각은 IClock으로 주입(기본 Time.time). 상태는 여전히 러너가 소유. 매니저 아님.

발동 = AbilityBeat

평타·Q·E 모두 애니메이션 이벤트가 발동 시점을 소유. 슬롯키 + 비트 인덱스로 통일.

연출 = 분리

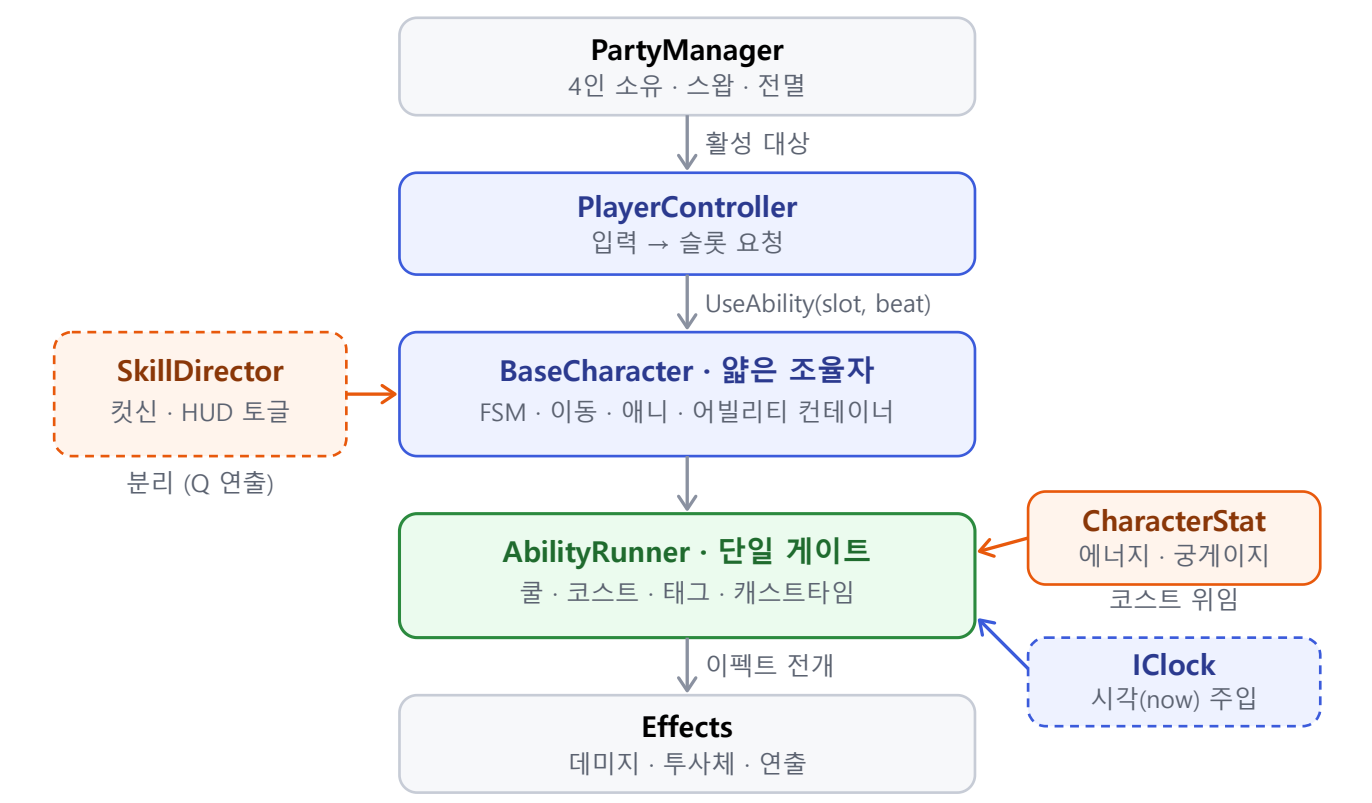
Timeline·HUD 토글은 캐릭터 밖 SkillDirector로. 캐릭터는 발동만, 연출은 디렉터가.

목표 한 줄

"캐릭터는 얇은 몸통 + 어빌리티 컨테이너, 발동 판단은 러너 하나, 연출은 밖." 새 캐릭터 = 프리팹 + 어빌리티 SO. 새 스킬 = Effect 조합. 새 클래스는 새 동사가 필요할 때만.

01 목표 아키텍처 — 누가 무엇을 소유하나

파란 척추 = 발동이 흐르는 몸통. 좌우로 빠진 것 = 캐릭터에서 떼어낸 관심사(연출·자원·시각).



파란 실선 = 발동 흐름 · 점선 박스 = 새로 분리/주입되는 부품 · 주황 = 캐릭터에서 떼어낸 관심사

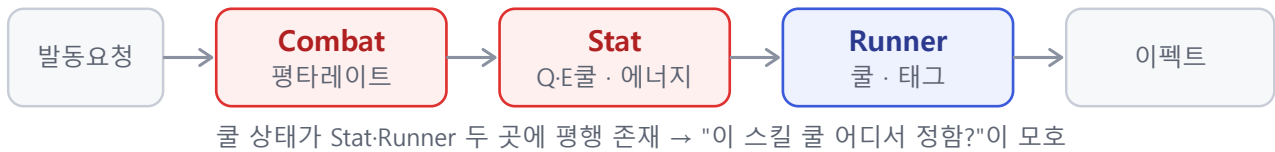
책임 재분배

주체	말을 것	변화
AbilityRunner	쿨·코스트·태그·캐스트 단일 게이트 + UI 읽기 API	유지·강화
CharacterStat	에너지·궁게이지 자원만. Q/E 쿨 타이머 제거	쿨 삭제
CharacterCombat	평타레이트 → AbilityData.Cooldown 함수. 사실상 소멸	해체
IClock	현재 시각 제공(기본 Time.time). 러너에 주입	신규(이음새)
SkillDirector	Timeline 재생 + 컷신 중 HUD 토글	신규(추출)
BaseCharacter	FSM·이동·애니·어빌리티 컨테이너 조율. 발동만	슬림화
PartyManager	스왑·전멸. TickCooldowns 제거(절대시각)	축소

02 핵심 변경 ① — 게이팅을 러너로 단일화

가장 크리티컬한 변경. 세 주인의 게이트를 러너 한 줄기로 합치면 비로소 "스킬 = asset 하나"가 성립한다.

AS-IS — 게이트 3주인 · 쿨 상태 2곳 중복



TO-BE — 러너 단일 게이트



평타레이트는 AbilityData.Cooldown으로 흡수 · 쿨 상태는 러너 한 곳 · 시각은 주입 · 코스트만 Stat에 위임

딱 하나 새로 이어야 할 이음새 — 코스트

에너지(자원)는 Stat에 남되, 소비 시점의 순서를 러너 게이트가 소유해야 "1 way"가 완성된다. 러너가 게이트 단계에서 `ctx.CasterStat` (자원 제공자)에게 "코스트 되나? → 소비해"를 호출하는 식. 지금

`AbilityData.Cost` 는 죽은 필드라 여기가 실제 배선 지점.

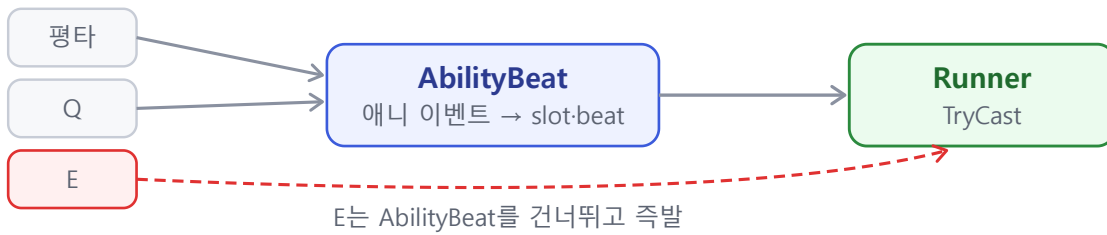
이 변경이 풀어주는 것

- ① 디자이너가 스킬 쿨/코스트를 **asset 한 곳**에서 정함 · ② 스왑아웃 파티원 쿨이 **절대시각으로 자동 진행** (TickCooldowns 불필요) · ③ 쿨감(CDR) 스탯을 러너가 `ctx` 로 곱함 · ④ UI는 러너의 읽기 API로 남은 쿨 조회.

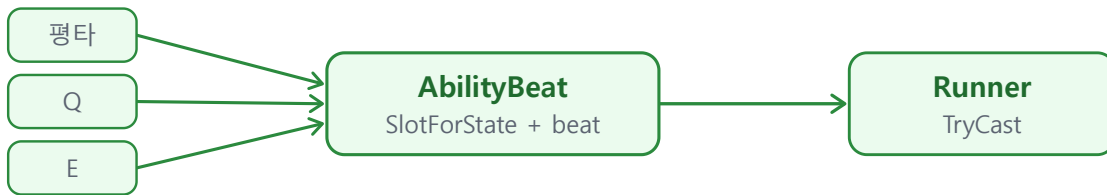
03 핵심 변경 ② 발동 경로 통일 · ③ 컷신 분리

E만 옆길로 즉발하던 것을 한 경로로. 그리고 캐릭터가 짙어진 연출을 밖으로.

AS-IS — E만 옆길(즉발, 프레임 타이밍 없음)



TO-BE — 평타·Q·E 모두 AbilityBeat 경유



E도 `SlotForState(E_Skill → Ex)` 한 줄 추가 + `UseSkillE`는 게이트·상태전환만. 발동 시점은 애니 이벤트가 소유

③ 컷신 / Q 를 밖으로

지금 `skillTimeline` 은 BaseCharacter의 컴포넌트라 Q가 "그 타임라인 틀기"에 고정되고, `OnSkillEnter/OnCutsceneEnded` 가 HUD 캔버스를 끄고 켜다(프레젠테이션 누수). 이를 **SkillDirector**로 추출한다:

- Timeline 재생·정지와 컷신 중 HUD 토글을 디렉터가 소유
- 컷신의 발사 프레임에서 시그널 → `UseAbility(Skill, beat)` — 다른 슬롯과 같은 beat 경로
- BaseCharacter는 타임라인 참조·캔버스 로직을 잃고 얹아짐 → "캐릭터 추가 = 데이터"에 근접

보스와 같은 철학

"Timeline = 언제·연출 / AbilitySystem = 무엇을" 분리하는 보스가 이미 쓰는 방식이다. 플레이어 Q를 여기에 정렬하면 세 진영의 발동이 완전히 같은 결로 수렴한다.

04 러너 게이트 순서 & 단계별 실행 계획

러너 내부의 단일 게이트가 밟는 순서, 그리고 안전하게 쪼갬 5단계 로드맵.

러너 단일 게이트의 순서

- 1 쿨다운 검사 — `IClock.Now < _cooldownEnd` 이면 아직 쿨 → 중단 (절대시각, tick 불필요)
- 2 코스트 검사·소비 — `ctx.CasterStat` 에 위임: 에너지 되나? 되면 소비. Q 공개이지가 여기
- 3 태그 게이트 — Required 전부 보유 & Blocked 하나도 없음 (기절 중 발동 불가 등)
- 4 쿨 시작 + 캐스트타임 — `_cooldownEnd = Now + Cooldown·CDR` . 채널링이면 대기
- 5 이펙트 순차 실행 — 컨텍스트와 함께 순서대로. GrantsTags는 try/finally 해제
- 6 1~3 실패 시 — 상태변화 없이 조용히 중단, 쿨·코스트 미소모(소모가 게이트 뒤)

단계별 실행 계획 — 각 단계 독립 배포·롤백 가능

PHASE 0 안전 이음새 (즉시)

IClock 도입(러너 생성자 주입, 기본 `UnityClock → Time.time`) — 동작 100% 불변. 동시에 독립 버그인 `PlayerController.Dispose` 이벤트 누수 수정(Register↔Unregister 대칭).

검증: 기존 동작 그대로 · GameScene 재진입 후 Q/E 중복 발동 0

PHASE 1 게이팅 단일화 (핵심)

Stat의 Q/E 쿨 타이머 → 러너로 이관 · Combat 평타레이트 → `AbilityData.Cooldown` · Stat은 에너지(자원)만 · 러너에 코스트 위임 seam + UI 읽기 API · PartyManager `TickCooldowns` 제거.

검증: 평타/Q/E 쿨 정상 · 스왑아웃 멤버 쿨 진행 · 쿨 UI 채움 정상

PHASE 2 발동 경로 통일

E를 AbilityBeat(Ex 슬롯)로 · `SlotForState` 에 `E_Skill → Ex` · `UseSkillE` 는 게이트·상태전환만. 세 슬롯 모두 애니 이벤트가 발동 시점 공유.

검증: E 투사체가 던지는 프레임에 맞춰 발사 · 세 슬롯 경로 동일

PHASE 3 컷신 / Q 분리

SkillDirector 추출(Timeline + HUD 토글) · BaseCharacter에서 캔버스·타임라인 로직 제거 · Q도 시그널 → beat 경로. God object 완화.

검증: Q 컷신·HUD 토글 정상 · BaseCharacter 라인 수 감소 · 새 캐릭터 배선 간소화

PHASE 4 (선택) 잔여 정리

입력 인텐트 → 슬롯 매핑 데이터화(핸들러 증식 차단) · IClock 선택시간 활용(몹 슬로우 등) 여지 · 죽은 필드/중복 상태(IsUsingSkill vs FSM) 정리.

검증: 새 슬롯 추가 시 코드 0 · 회귀 없음

착수 순서

Phase 0 → 1이 실질 1단계다. 0은 리스크 0(이음새+버그수정)이라 지금 바로, 1은 이 청사진의 심장. 2·3은 1 위에서 자연스럽게 따라오고, 4는 여유 있을 때. 각 단계가 스스로 배포 가능해 중간에 멈춰도 안전하다.